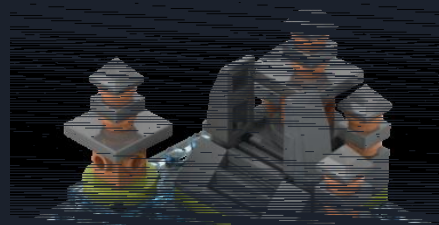
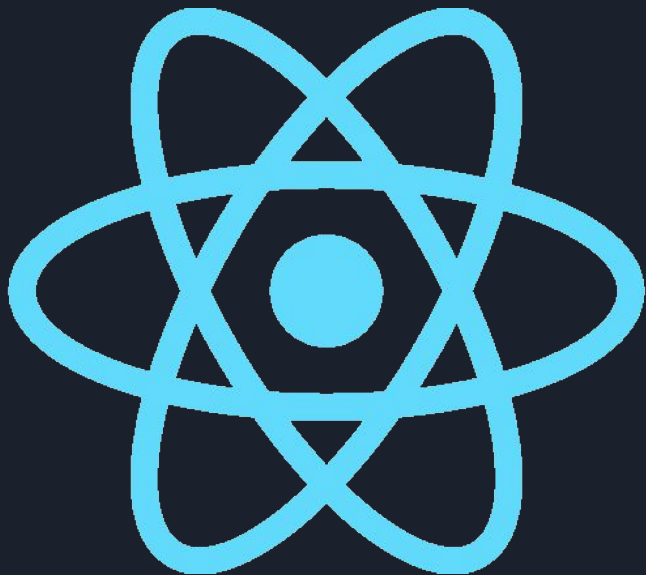




REACT PARTE 2

COMPONENTES Y PROPS





OBJETIVOS DE LA SESIÓN

- Comprender el concepto de componentes reutilizables en React
- Aprender a crear y utilizar props para pasar datos entre componentes
- Implementar renderizado condicional en React
- Crear listas dinámicas de componentes
- Desarrollar una aplicación práctica integrando todos los conceptos



AGENDA

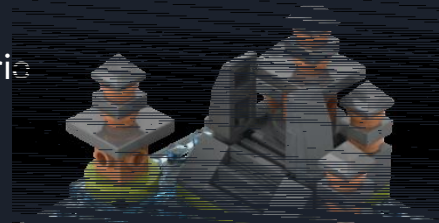
Contenidos del Día

- **Parte 1:** Componentes y reutilización
- **Parte 2:** Props y paso de datos
- **Parte 3:** Renderizado condicional y listas
- **Práctica Final:** Lista de componentes dinámica

¿QUÉ SON LOS COMPONENTES?

Concepto de Componentes

- Los componentes son ****piezas reutilizables**** de interfaz de usuario
- Cada componente encapsula su propia lógica y presentación
- Permiten dividir la UI en piezas independientes y reutilizables





Beneficios de los componentes

¿Por qué usar componentes?

1. **Reutilización:** Escribir una vez, usar en muchos lugares
2. **Mantenimiento:** Cambios en un lugar afectan todos los usos
3. **Organización:** Código más limpio y estructurado
4. **Colaboración:** Diferentes desarrolladores pueden trabajar en componentes distintos
5. **Testing:** Probar componentes de forma aislada

Anatomía de un componente



```
1 function MiComponente() {  
2   return (  
3     <div>  
4       <h1>¡Hola desde mi componente!</h1>  
5       <p>Este es contenido del componente</p>  
6     </div>  
7   );  
8 }
```

Un componente React es simplemente una **función** que devuelve JSX.

Reglas importantes:

- El nombre debe empezar con **mayúscula**
- Debe devolver **un solo elemento** (o Fragment)
- Puede contener JavaScript dentro de llaves `{}`



Componente saludo

```
// Componente simple
function Saludo() {
  return (
    <div className="saludo">
      <h2>¡Bienvenido!</h2>
      <p>Es genial tenerte aquí</p>
    </div>
  );
}
```

```
// Usar el componente en App
function App() {
  return (
    <div>
      <Saludo />
      <Saludo />
      <Saludo />
    </div>
  );
}
```



EJERCICIO PRÁCTICO - PRIMER COMPONENTE

-Práctica Guiada: Tu Primer Componente

Crear un componente `TarjetaPersona` que muestre:

1. Un nombre (por ahora hardcodeado)
2. Una profesión
3. Una imagen de avatar. Te recomiendo que si quieres utilizar un imagen de tu PC la guardar en la carpeta public y la llames `src= './título.jpg'`

```
function TarjetaPersona() {  
  return (  
    <div className="tarjeta">  
        
      <h3>Juan Pérez</h3>  
      <p>Desarrollador Web</p>  
    </div>  
  );  
}
```




Limitaciones sin Props

El problema de los componentes estáticos

Nuestros componentes siempre muestran la misma información

¿Qué pasa si queremos mostrar diferentes personas?

- ¿Crear un componente para cada persona?
- ¿Copiar y pegar código?

****Solución:** → Props - La forma de hacer componentes dinámicos

```
function TarjetaPersona() {  
  return (  
    <div>  
      <h3>Juan Pérez</h3> { /* Siempre "Juan Pérez" */}  
      <p>Desarrollador</p> { /* Siempre "Desarrollador" */}  
    </div>  
  );  
}
```

Introducción a las Props

Características de los Props:

- ✓ Solo lectura - No puedes modificarlos
- ✓ Vienen de arriba - Los recibe del componente padre
- ✓ Son permanentes - No cambian dentro del componente

```
1 function PropsTarjetaPersona(props) {  
2   return (  
3     <div className="tarjeta">  
4       <img src={props.imagen} alt="Avatar" />  
5       <h3>{props.nombre}</h3>  
6       <p>{props.profesion}</p>  
7     </div>  
8   );  
9 }  
10 export default PropsTarjetaPersona;
```

Sintaxis básica de props

Llamamos a las props que queremos en nuestro App.jsx

Reutilizamos el componente pero con diferentes props.

TIP: Para comentar y descomentar una sección de código. (seleccionamos sección)

- ctrl + K + C (Comentar)
- ctrl + K + U (Descomentar)

```
1  function App() {  
2    return (  
3      <>  
4        <PropsTarjetaPersona  
5          nombre= "Jorge"  
6          profesion= 'Albañil'  
7          imagen= './Denver.jpg'  
8        />  
9        <PropsTarjetaPersona  
10         nombre= "Sara"  
11         profesion= 'Carpintera'  
12         imagen= './hola.jpg'  
13       />  
14       <PropsTarjetaPersona  
15         nombre= "Alberto"  
16         profesion= 'profesor'  
17         imagen= './hola2.jpg'  
18       />  
19     </>  
20   )  
21 }
```

Destructuring de props



```
1  function PropsTarjetaPersona({imagen, nombre, profesion}) {  
2    return (  
3      <div className="tarjeta">  
4        <img src={imagen} alt="Avatar" />  
5        <h3>{nombre}</h3>  
6        <p>{profesion}</p>  
7      </div>  
8    );  
9  }  
10 export default PropsTarjetaPersona;
```

Props por defecto

1. Botones

```
function Boton({ texto = "Enviar" }) {  
  return <button>{texto}</button>;  
}  
  
// Útil porque no siempre querrás escribir el texto
```

2. Imagen por default por si hay imagen

```
jsx  
  
function Avatar({ src = "../default-avatar.png", nombre = "Usuario" }) {  
  return <img src={src} alt={nombre} />;  
}  
  
// Súper útil: si falla la imagen, hay una por defecto
```



EJERCICIO PRÁCTICO: Tarjeta de Producto Amazon

Instrucciones:

Crear un componente

TarjetaProducto.jsx que reciba las siguientes props:

- imagen - URL de la imagen del producto
- nombre - Nombre del producto
- precio - Precio del producto

Como debería verse:

```
// En App.jsx  
<TarjetaProducto  
  imagen="./iphone.jpg"  
  nombre="iPhone 15 Pro"  
  precio="1,199€"  
>  
</>
```

Requisitos:

Crear archivo TarjetaProducto.jsx

Usar las props para mostrar la información

Aplicar algún estilo CSS básicos (imagen redondeada) Crear e importar archivo css.

Hacer que sea reutilizable

Renderizado de condicionales

Conceptos clave:

A veces queremos mostrar elementos solo en ciertas situaciones

React nos permite mostrar/ocultar contenido dinámicamente

Dos técnicas principales: Operador Ternario y Operador &&

```
// Operador Ternario: Siempre muestra algo  
{condición ? <ElementoA /> : <ElementoB />}
```

```
// Operador &&: Muestra algo o nada  
{condición && <Elemento />}
```

Ejemplos visuales:

-  "Usuario conectado" vs  "Usuario desconectado"
-  "Tienes 3 notificaciones" (solo si hay notificaciones)

Renderizado de condicionales en JSX (REACT)

```
1 function PanelAdmin({ esAdmin }) {
2   return (
3     <div>
4       <h2>Panel de Control</h2>
5
6       <button>Ver estadísticas</button>
7
8       {/* Solo si eres admin puedes utilizar este boton */}
9
10      {esAdmin && (
11        <button style={{ backgroundColor: 'red' }}>
12          Eliminar usuarios
13        </button>
14      )}
15    </div>
16  );
17 }
```




Renderizado de Listas

Creando listas dinámicas

Para mostrar listas de elementos usamos el método `.map()`:

- Recorre un array y transforma cada elemento, devolviendo un nuevo array. Veamos algunos ejemplos en JS

```
// Array de nombres
const nombres = ['Ana', 'Carlos', 'María'];
const saludos = nombres.map((nombre) => `Hola ${nombre}`);
// Resultado: ['Hola Ana', 'Hola Carlos', 'Hola María']

// Array de objetos
const personas = [
  { nombre: 'Juan', edad: 25 },
  { nombre: 'Ana', edad: 30 }
];
const soloNombres = personas.map((persona) => persona.nombre);
// Resultado: ['Juan', 'Ana']
```

Renderizado de Listas

Creando listas dinámicas



```
1 function ListaCompras({ objetos }) {  
2   return (  
3     <div>  
4       <h3>Lista de Compras</h3>  
5       <ul>  
6         {objetos.map((objeto) => (  
7           <li>{objeto}</li>  
8         ))}  
9       </ul>  
10  
11     </div>  
12   );  
13 }
```



```
1 function App() {  
2   const compras = ["Leche", "Pan", "Huevos", "Manzanas"]; // Array aquí  
3  
4   return (  
5     <div>  
6       <ListaCompras objetos={compras} /> {/* Se pasa como prop */}  
7     </div>  
8   );  
9 }
```



```
1 function ListaCompra2 ({objetos}) {
2
3   return(
4     <>
5     <h2>Lista de la compra</h2>
6     <ul>
7       {objetos.map((objeto) => (
8         <li key={objeto.id}>
9           <span>{objeto.nombre}</span>
10          <span>{objeto.image}</span>
11        </li>
12      )})
13    </ul>
14    </>
15  )
16 };
```



```
1 function App() {
2   const compra3 = [
3     {id: 1, nombre:"Ensalada", image: '🥗'},
4     {id:2, nombre: "Cerveza", image: '🍺'},
5     {id:3, nombre: "Patatas", image: '🍟'}
6   ];
7   return (
8     <>
9     <ListaCompra2 objetos={compra3} />
10    </>
11  )
12 }
```

LA PROP KEY EN LISTAS

Importancia de la prop key

¿Por qué necesitamos `key`?

- React usa las keys para identificar qué elementos han cambiado
- Mejora el rendimiento al re-renderizar listas
- Evita bugs raros en componentes con estado

Cuando utilizas `map` lo estás utilizando para devolver algo como un `div`, `li`, `span`, etc. Si no tiene un ID identificativo, React se queja. El código va a funcionar pero te va a dar un warning.

✓ Correcto:

```
{usuarios.map(usuario
=> (
  <TarjetaUsuario
    key={usuario.id}
    // ID único del
    usuario
    nombre=
    {usuario.nombre}
  />
))}
```

Lista de componentes

Aparte de renderizar arrays y arrays de objetos, podemos renderizar múltiples componentes a través `.map()`

Es decir `.map()` transforma cada objeto del array en un componente React.

Tenemos un array de objetos que serían las personas de nuestro equipo.

```
1 const equipo = [  
2   { id: 1, nombre: "Ana García", profesion: "Desarrolladora", edad: 28 },  
3   { id: 2, nombre: "Carlos López", profesion: "Diseñador", edad: 32 },  
4   { id: 3, nombre: "María Rodríguez", profesion: "Project Manager", edad: 29 }  
5 ];
```

```
1 function ListaPersonas({ personas }) {  
2   return (  
3     <div className="lista-personas">  
4       <h2>Nuestro Equipo</h2>  
5       <div className="grid">  
6         {personas.map((persona) => (  
7           <TarjetaPersona  
8             key={persona.id}  
9             nombre={persona.nombre}  
10            profesion={persona.profesion}  
11            edad={persona.edad}  
12          />  
13        ))}  
14       </div>  
15     </div>  
16   );  
17 };  
18 export default ListaPersonas;
```

Combinando conceptos

Lista de tareas.jsx

Organización de archivos:

- [tareas.js](#) (assets): Array de objetos con estructura `{id, texto, completada}`
- ListaTareas.jsx: Componente que recibe y renderiza las tareas
- App.jsx → Componente padre que importa datos y componente

Nuevo método: `.filter()`

- **Propósito**: Filtrar elementos de un array según una condición
- **Diferencia con `.map()`**:
 - `.map()` → transforma **TODOS** los elementos
 - `.filter()` → selecciona **ALGUNOS** elementos que cumplen condición

Flujo de datos:

1. **App.jsx** importa las tareas desde assets
2. **App.jsx** pasa las tareas como props a ListaTareas
3. **ListaTareas** recibe y procesa los datos

Children Props

`children` es una prop especial que contiene el contenido entre las etiquetas del componente:

Piensa en un componente como una **caja decorativa**:

- La caja tiene un diseño fijo (el JSX del componente)
- Pero puedes meter **diferentes cosas dentro** (children)
- La caja se adapta a lo que metas dentro

```
1 // 1. Definimos el componente "caja"
2 function Tarjeta({ children, titulo }) {
3   return (
4     <div className="tarjeta">
5       <h3>{titulo}</h3>
6       <div className="contenido">
7         {children} {/* Aquí va lo que pongas entre las etiquetas */}
8       </div>
9     </div>
10  );
11 }
12
13 // 2. Usamos la "caja" y metemos cosas dentro
14 <Tarjeta titulo="Mi Perfil">
15   {/* Todo esto de aquí es "children" */}
16   <p>Nombre: Juan Pérez</p>
17   <p>Email: juan@email.com</p>
18   <button>Editar Perfil</button>
19 </Tarjeta>
```

Ejercicio práctico

Paso 1: Preparar datos

Crea `src/assets/tareas.js` con el array de 8 tareas proporcionado

Usa `export default tareas`

Paso 2: Componente ListaTarea

Recibe props: `{ tareas, mostrarCompletadas }`

Aplica `.filter()` con la condición:
`mostrarCompletadas || !tarea.completada`

Usa `.map()` para renderizar cada tarea en `<li>`

Paso 3: Estilos condicionales

`textDecoration`: `line-through` si completada, none si no

`color`: `gray` si completada, `black` si no

Mostrar  solo en tareas completadas usando `&&`

Paso 4: Conectar en App.jsx

Importa las tareas y el componente `ListaTareas`

Pasa las props: `tareas={tareas}` y `mostrarCompletadas={true/false}`

 Resultado esperado:

Con `mostrarCompletadas={false}`: Solo tareas pendientes

Con `mostrarCompletadas={true}`: Todas las tareas (completadas tachadas)



Resultado esperado

Mis Tareas

- Comprar leche y pan
- ~~Estudiar React y props~~ ✓
- Hacer ejercicio 30min
- ~~Leer documentación de JavaScript~~ ✓
- Preparar presentación del proyecto
- Llamar al dentista
- ~~Organizar escritorio~~ ✓
- Revisar emails pendientes



Flujo de Datos en React

COMUNICACIÓN ENTRE COMPONENTES

Regla fundamental: Los datos fluyen hacia abajo (de padre a hijo)

App (datos tareas)

↓ props

ListaTareas (recibe y filtra datos)

↓ renderiza cada tarea

 individual (muestra datos)

```
// App.jsx - Componente padre con los datos
function App() {
  const tareas = [...]; // Array de tareas desde assets

  return (
    <div>
      <h1>Mi App de Tareas</h1>
      <ListaTareas
        tareas={tareas}
        mostrarCompletadas={true}
      />
    </div>
  );
}
```

```
// ListaTareas.jsx - Recibe datos y los procesa
function ListaTareas({ tareas, mostrarCompletadas }) {
  const tareasFiltradas = tareas.filter(tarea =>
    mostrarCompletadas || !tarea.completada
  );

  return (
    <div>
      <h3>Mis Tareas</h3>
      <ul>
        {tareasFiltradas.map(tarea => (
          <li
            key={tarea.id}
            style={{
```

Mejores prácticas

Buenas prácticas con componentes

1. Nombres descriptivos:

jsx//  Bueno

```
function ListaTareas({ tareas, mostrarCompletadas }) { ... }
```

//  Evitar

```
function Lista({ items, show }) { ... }
```

2. Una responsabilidad por componente:

jsx//  Cada componente tiene una función clara

App → Maneja datos

ListaTareas → Filtra y renderiza lista

ItemTarea → Muestra una tarea individual

4. Keys únicas en listas:

jsx//  Correcto

```
{tareasFiltradas.map(tarea => (  
  <li key={tarea.id}> { /* ID único */ }  
  {tarea.texto}  
</li>  
)}
```

//  Evitar

```
{tareasFiltradas.map((tarea, index) => (  
  <li key={index}> { /* Índice como key */ }  
  {tarea.texto}  
</li>  
)}
```

Aplicación Completa: Lista de Contactos

1. En el moodle de KC encontrareis las instrucciones para realizar el proyecto
2. Fecha límite: 4 de julio
3. Forma de entrega: URL de netlify o github

 Tecnologías a Usar: React + Vite

- Componentes funcionales
- Props y renderizado condicional
- CSS responsive

A tener en cuenta:

Funcionalidad correcta (sin errores)

Uso correcto de componentes y props

Diseño responsive y profesional

Despliegue exitoso en Netlify





Debugging y herramientas

React Developer Tools:

- Extensión del navegador para inspeccionar componentes
- Permite ver props, estado y jerarquía de componentes

Errores comunes

1. Olvidar la prop key: `* `Warning: Each child in a list should have a unique "key" prop``
2. Props mal nombradas: Verificar que los nombres coincidan
3. Datos undefined: Usar valores por defecto

Para ver errores, es recomendable hacer `console.log()` para ver si se están pasando los datos correctamente.



```
1 function MiComponente({ datos = [] }) {  
2   console.log('Props recibidas:', { datos }); // Para debug
```



Recursos adicionales

Documentación oficial:

- [React Docs - Componentes y Props](<https://react.dev/learn/passing-props-to-a-component>)
- [React Docs - Renderizado de Listas](<https://react.dev/learn/rendering-lists>)
- [React Docs - Renderizado Condicional](<https://react.dev/learn/conditional-rendering>)

Herramientas:

- React Developer Tools (Chrome/Firefox extension)
- [CodeSandbox](<https://codesandbox.io>) - Para practicar online
- [React Fragment](<https://react.dev/reference/react/Fragment>) - Para evitar divs innecesarios

Ejercicios para casa:

- Proyecto Lista de contactos
- Proyecto Equipos Departamentos